Got a problem statement for which we need to build a website for real-life usecase that can be easily used by non technical users, handles realistic values and scenarios & could be easily deployed and without major changes (it's actually for a promptathon). For now, do not generate the code, just think and evaluate about all the needs, requirements, architecture, design, ui, ux, frontend, backend, data management, security, monitoring etc. Specially consider that it should work flawlessly, user-friendly & also keep the judges pov in mind that what should they judge for or look at and accordingly focus on overall enhancement of the website. And need to focus on strategic modular programming

Show more

This is actually a very good Promptathon problem because it looks simple at first glance, but the judges will likely evaluate much more than just "a Kanban board that moves cards."

The teams that simply build 4 columns and drag-and-drop candidates will probably score average.

The teams that build a realistic ATS workflow with robust business rules, validation, UX, data integrity, and modular architecture will stand out.

1. Understanding The Real Problem

This is not a Kanban board problem.

It is an **Applicant Tracking System (ATS)** with:

- Workflow management
- Candidate management
- Business rule enforcement
- State machine logic
- Recruitment analytics
- Data integrity

The Kanban board is only the UI layer.

The real challenge is:

> Build a system that prevents invalid recruitment decisions.

2. What Judges Will Likely Evaluate

Most judges subconsciously score these categories:

Area	Weight
Functional correctness	High
Rule enforcement	Very High
UX	High
Reliability	High
Real-world applicability	High
Architecture	Medium-High
Visual polish	Medium

Area	Weight
Code quality	Medium
Edge case handling	Very High

The "Twist" section is probably where most teams will lose points.

3. Core Entities

Candidate

```
id
name
rollNumber
roleApplied
stage
technicalScore
statusLocked
createdAt
updatedAt
notes
```

Example

```
Name:
Ashutosh Sahu

Roll:
CSE24123

Role:
Frontend Developer

Stage:
Interviewing

Technical Score:
null

Locked:
false
```

4. Pipeline Architecture

The ATS pipeline is actually:

```
Applied
↓
Interviewing
↓
Technical Test
↓
Offered
```

Alternative path:

```
Applied
↓
Interviewing
↓
Technical Test
↓
Rejected
```

And

```
Rejected
↓
LOCKED FOREVER
```

5. Important Design Decision

Do NOT let the UI control rules.

Instead:

```
UI
↓
Stage Manager
↓
Validation Layer
↓
Business Rules Engine
↓
Data Update
```

This is much safer.

6. Suggested Architecture

```
Frontend
│
├── Components
│   ├── Board
│   ├── Column
│   ├── CandidateCard
│   ├── Dialogs
│   └── Forms
│
├── Services
│   ├── CandidateService
│   ├── PipelineService
│   └── ValidationService
│
├── State
│   └── ATSStore
│
└── Utils
```

Very modular.

Judges love modularity.

7. State Machine Thinking

This is where many teams fail.

Treat stages as a State Machine.

Allowed transitions:

```
Applied
↔ Interviewing

Interviewing
↔ Applied

Interviewing
→ Technical Test

Technical Test
→ Offered

Technical Test
```

```
→ Rejected

Rejected
→ NO WHERE
```

Any invalid transition:

```
Rejected → Offered

Rejected → Applied

Rejected → Interviewing
```

Must fail instantly.

8. Candidate Card Design

Keep it information dense.

Card

```
_____________
Ashutosh Sahu

CSE24123

Frontend Developer

Stage Badge

Score (if exists)

Created Date
_____________
```

Optional:

Small avatar circle using initials.

```
AS
```

Looks professional.

9. Add Candidate Flow

A polished ATS should have:

Required

- Name
- Roll Number
- Applying Role

Validation

Name:

```
Minimum 2 chars
```

Roll Number:

```
Unique
```

Role:

```
Required
```

10. Technical Test Workflow

This is the heart of the challenge.

When user moves candidate to:

```
Technical Test
```

System MUST:

```
Open Score Dialog
```

Example:

```
Enter Technical Score

[ 82 ]

Cancel
Submit
```

Validation:

```
0 <= score <= 100
```

Only then proceed.

11. Auto-Rejection Engine

Required rule:

```
Score < 70
```

Immediately:

```
Stage = Rejected
Locked = True
```

No user decision.

No confirmation.

No override.

No edit.

No exception.

This is the most important business rule.

12. Lock Mechanism

Every candidate should have:

```
isLocked
```

When rejected:

```
isLocked = true
```

Before every movement:

```
if locked:
```

```
reject action
```

Display visual badge:

```
🔒 Locked
```

13. Offered Confirmation

When moving:

```
Technical Test
→ Offered
```

Show:

```
Offer Candidate?

Ashutosh Sahu
Score: 88

Cancel
Confirm
```

Only after confirmation:

```
stage = Offered
```

14. UX Enhancements Judges Will Notice

Stage Counters

```
Applied (12)

Interviewing (4)

Technical Test (2)

Offered (5)

Rejected (8)
```

Instant professional feel.

Search Bar

```
Search by:
- Name
- Roll Number
```

Very useful.

Role Filter

```
All

Frontend

Backend

Design

Marketing
```

Sort

```
Newest

Oldest

Highest Score
```

15. Analytics Section

Small dashboard at top.

```
Total Candidates

Accepted Rate

Rejected Rate

Average Score

Offers Made
```

This gives judges:

> "This is a real ATS"

feeling.

16. Error Prevention

Disable actions before errors occur.

Example:

Instead of allowing:

```
Move To Offered
```

Disable button if:

```
No technical score
```

Much better UX.

17. Notifications

Use toast system.

Examples:

```
✓ Candidate moved

✓ Score recorded

✗ Candidate locked

✗ Invalid transition
```

18. Data Persistence

For Promptathon:

LocalStorage

Enough.

Store:

```
Candidates
```

Automatically.

On refresh:

```
Restore board
```

Judges love seeing persistence.

19. Edge Cases To Handle

Duplicate Roll Number

Reject.

Score = 70

Allowed.

```
70+
```

passes.

Score = 69

Auto reject.

Empty Score

Reject submission.

Negative Score

Reject.

Above 100

Reject.

Offered Candidate

Can move backward?

You must decide.

Recommended:

```
Yes
```

to Technical Test.

More flexible.

Rejected Candidate

Never move.

Ever.

20. UI Design Recommendation

Modern ATS style.

Layout

```
Header

Analytics Cards

Search + Filters

Kanban Board
```

Colors

Applied

```
Blue
```

Interviewing

```
Orange
```

Technical Test

```
Purple
```

Offered

```
Green
```

Rejected

```
Red
```

Easy visual recognition.

21. Strategic Modular Programming Structure

If I were building this for Promptathon:

```
ATS Engine
│
├── Candidate Module
│
├── Stage Transition Module
│
├── Validation Module
│
├── Scoring Module
│
├── Locking Module
│
├── Analytics Module
│
```

```
├── Storage Module
│
└── UI Components
```

Every module has one responsibility.

This is exactly the kind of architecture judges like to hear during evaluation.

Final Evaluation

A **minimum-pass solution** is:

- ✅ Add candidate
- ✅ Move cards
- ✅ Score dialog
- ✅ Auto reject
- ✅ Confirmation dialog

A **high-scoring Promptathon solution** is:

- ✅ Everything above
- ✅ State-machine based transitions
- ✅ Validation engine
- ✅ Locked candidate system
- ✅ Search/filter/sort
- ✅ Analytics dashboard
- ✅ LocalStorage persistence
- ✅ Toast notifications
- ✅ Professional ATS UI
- ✅ Modular architecture
- ✅ Strong edge-case handling

If executed cleanly, this is realistically a project that could be used by a college club, placement cell, or student organization with almost no changes after deployment. 🚀

index(27).html
File

script(21).js
JavaScript

style(23).css
File

do the final audit and check everything , and the add candidate button isn't working for sure and check if all the prompthon requirements are fufilled or not also if there are problem then be specific and show exactly where to do the fix

index(28).html
File

script(22).js
JavaScript

style(24).css
File

this is it, now have a final look

That's actually a very useful clue. 👍

If:

```
View
↓
Details Modal Opens
```

but

```
❌ X button does nothing
```

then your **modal system is working**, but the **close button isn't connected to ModalManager**.

Check 1

Find the X button inside your Details Modal.

It probably looks like:

HTML

```
<button
    id="closeDetailsModal"
    class="modal-close-btn">
    ×
</button>
```

or similar.

Fix

Add:

HTML

```
data-close-modal
```

so it becomes:

HTML

```
<button
    id="closeDetailsModal"
    class="modal-close-btn"
    data-close-modal>
    ×
</button>
```

Why?

Part 5 uses:

JavaScript

```
document
    .querySelectorAll(
        "[data-close-modal]"
    )
```

to automatically wire close buttons.

Without:

HTML

```
data-close-modal
```

the button is just decorative.

Check Every Modal

Make sure ALL these buttons have:

HTML

```
data-close-modal
```

Candidate Modal

HTML

```
id="closeCandidateModal"
```

Details Modal

HTML

```
id="closeDetailsModal"
```

Technical Score Modal

HTML

```
id="closeTechnicalScoreModal"
```

Offer Modal

HTML

```
id="closeOfferModal"
```

(if present)

Delete Modal

HTML

```
id="closeDeleteModal"
```

(if present)

Cancel Buttons

HTML

```
cancelCandidateBtn
cancelTechnicalScoreBtn
cancelOfferBtn
cancelDeleteBtn
```

Quick Test

After adding:

HTML

```
data-close-modal
```

1. Refresh
2. Click View
3. Click X

Expected:

```
Details Modal
↓
Closes
↓
Overlay disappears
```

One More Thing

If you already added `data-close-modal` and it still doesn't close, then the button may be **outside the modal content structure** or the modal event registration isn't running.

In that case, paste the HTML for the Details Modal (from `<div id="candidateDetailsModal"...>` to its closing `</div>`), and I can pinpoint the exact issue.

also why removed arrows with texts like back and next, give me fixes for both of the problems show exactly how to solve it and be specific

I can explain both issues exactly.

🚨 Problem 1: X Button Doesn't Close Modal

Why it's happening

Your Part 5 close system only works for buttons that contain:

HTML

```
data-close-modal
```

and then automatically runs:

JavaScript

```
ModalManager.closeAllModals();
```

If your Details Modal X button looks like:

HTML

```
<button
    id="closeDetailsModal"
    class="modal-close-btn">
    ×
</button>
```

then it is **not connected**.

Exact Fix

Find:

HTML

```
id="closeDetailsModal"
```

and change it to:

HTML

```
<button
    id="closeDetailsModal"
    class="modal-close-btn"
    data-close-modal>
    ×
</button>
```

Also check these buttons

Make sure ALL of them contain:

HTML

```
data-close-modal
```

HTML

```
closeCandidateModal
closeTechnicalScoreModal
closeDetailsModal
cancelCandidateBtn
cancelTechnicalScoreBtn
cancelOfferBtn
cancelDeleteBtn
```

🚨 Problem 2: Arrows Disappeared

This one is easy.

When we rebuilt the action buttons we changed them to:

HTML

```
View
Back
Next
Delete
```

and removed the icons.

Find inside candidateCardTemplate

You currently have something like:

HTML

```
<div class="candidate-card-actions">

    <button
        class="btn-action"
        data-action="view">

        View

    </button>

    <button
        class="btn-action"
        data-action="backward">

        Back

    </button>

    <button
        class="btn-action"
        data-action="forward">
```

Replace with

HTML

```
<div class="candidate-card-actions">

    <button
        class="btn-action"
        data-action="view"
        type="button">

        👁 View

    </button>

    <button
        class="btn-action"
        data-action="backward"
        type="button">

        ← Back

    </button>

    <button
        class="btn-action"
```

⭐ Better Promptathon Version

Instead of emojis:

HTML

```
← Back
Next →
```

use:

HTML

```
&#8592; Back
Next &#8594;
```

Example:

HTML

```
<button
    class="btn-action"
    data-action="backward"
    type="button">

    &#8592; Back

</button>

<button
    class="btn-action"
    data-action="forward"
    type="button">

    Next &#8594;

</button>
```

This looks cleaner and more professional.

One More Improvement (Recommended)

Currently every card always shows:

```
View
Back
Next
Delete
```

Even for:

```
Applied
```

where Back makes no sense.

And for:

```
Rejected
```

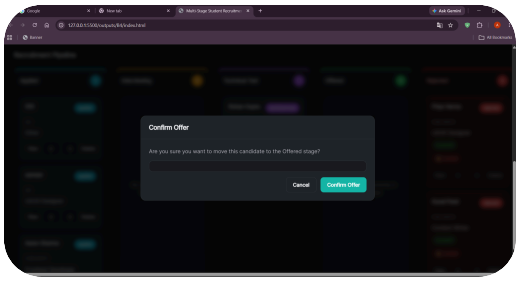
where Next shouldn't exist.

For Promptathon judges, a stronger UX is:

Stage	Show Buttons
Applied	View, Next, Delete
Interviewing	View, Back, Next, Delete
Technical Test	View, Back, Next, Delete
Offered	View, Back, Delete

Stage	Show Buttons
Rejected	View only

If you want, I can show the **exact JS change inside** `CandidateCardRenderer.create()` to make buttons automatically hide/show based on candidate stage. That would be a noticeable UX upgrade for judges. 🚀



we have to remove this thing that looking like a text area

Yep, I can see it in the screenshot.

That dark rounded rectangle under:

```
Are you sure you want to move this candidate to the Offered stage?
```

is almost certainly an **empty input/form control** that accidentally inherited your form styling.

Find the Offer Confirmation Modal

Search your HTML for:

HTML

```
id="offerConfirmationModal"
```

You should find something similar to:

HTML

```
<div id="offerConfirmationModal">
    <div class="modal-body">
        <p>
            Are you sure you want to move this candidate to the Offered stage?
        </p>
        <!-- Something here -->
    </div>
</div>
```

Most Likely Cause

You probably have something like:

HTML

```
<div class="form-group">
</div>
```

or

HTML

```
<input>
```

or

HTML

```
<div class="control-input">
</div>
```

inside the Offer Modal.

That empty element is rendering as the black textarea-looking box.

Quick Fix

Inside the Offer Modal, keep only:

HTML

```
<div class="modal-body">

    <p>
        Are you sure you want to move this candidate to the Offered stage?
    </p>

</div>
```

Nothing else.

If You See This

HTML

```
<div class="form-group">

</div>
```

Delete it completely.

If You See This

HTML

```
<input
    type="text">
```

Delete it completely.

If You See This

HTML

```
<div class="control-input"></div>
```

Delete it completely.

What the final Offer Modal body should look like

HTML

```
<div class="modal-body">

    <p>
        Are you sure you want to move this candidate to the Offered stage?
    </p>

</div>

<div class="modal-actions">

    <button
        id="cancelOfferBtn"
        class="btn btn-secondary"
        data-close-modal>

        Cancel

    </button>

    <button
        id="confirmOfferBtn"
        class="btn btn-primary">
```

After that, the weird textbox-looking area will disappear and the modal will look much cleaner and more professional for judges. 🚀